```
0001 setlocal_WC_0                          a@0
0003 putself
0004 putobject                              "a = "
0006 getlocal_WC_0                          a@0
0008 dup
0009 objtostring                            <calldata!mid:to_s, argc:0,
FCALL|ARGS_SIMPLE>
0011 anytostring
0012 putobject                              "."
0014 concatstrings                          3
0016 opt_send_without_block                 <calldata!mid:puts, argc:1,
FCALL|ARGS_SIMPLE>
0018 leave
```

For a full list of the opcodes, print out RubyVM::INSTRUCTION_NAMES.

Marshaling and Distributed Ruby

Ruby features the ability to *serialize* objects, letting you store them somewhere and reconstitute them when needed. You can use this facility, for instance, to save a tree of objects that represent some portion of application state—a document, a CAD drawing, a piece of music, and so on.

Ruby calls this kind of serialization *marshaling* (think of railroad marshaling yards where individual cars are assembled in sequence into a complete train, which is then dispatched somewhere). Saving an object and some or all of its components is done using the method dump. Typically, you will dump an entire object tree starting with some given object. Later, you can reconstitute the object using load.

Here's a short example. We have a class Chord that holds a collection of musical notes. We'd like to save away a particularly wonderful chord so we can e-mail it to a couple hundred of our closest friends so they can load it into their copy of Ruby and savor it too. Let's start with the classes for Note and Chord:

ospace/chord.rb

```
Note = Struct.new(:value) do
  def to_s
    value.to_s
  end
end

class Chord
  def initialize(arr)
    @arr = arr
  end

  def play
    @arr.join("-")
  end
end
```

Now we'll create our masterpiece and use Marshal.dump to save a serialized version to disk:

ospace/chord.rb

```
c = Chord.new(
  [
```